

# Evalify

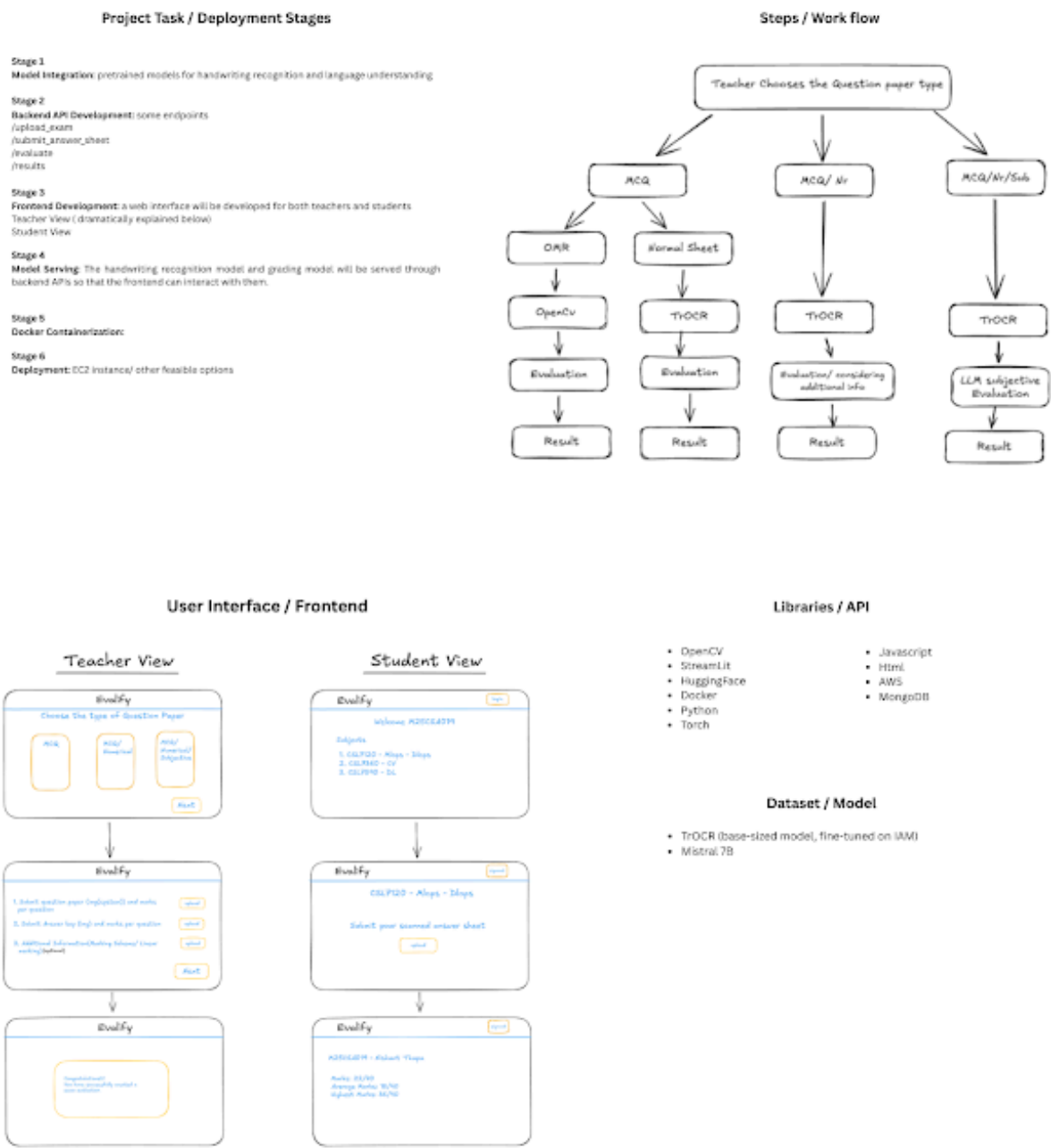
ML-DL-Ops (CSL 7120)

Nishant Thapa: M25CSA019 and Pranav: M25CSA021

IIT Jodhpur, Semester 2

April 19, 2026

## 1. Project Poster



## 2. Purpose of project

Evalify is an automated handwritten answer sheet evaluation system that removes the manual grading burden on teachers and TAs. It supports MCQ grading via OMR detection, numerical evaluation with tolerance-based logic, and subjective answer assessment using a locally hosted LLM(Ollama). Built with React, FastAPI, Docker, and MLflow, it aims to deliver fast, consistent, and scalable grading, freeing educators to focus on teaching instead of paperwork.

### Feedback Suggested:

Subjective answer evaluation using LLM needs more detail on prompt engineering and grading rubric handling. MongoDB choice could be justified better. Consider adding experiment tracking (MLflow) and model versioning details.

## 3. Project Phases and component in details

Project is completed in following phases

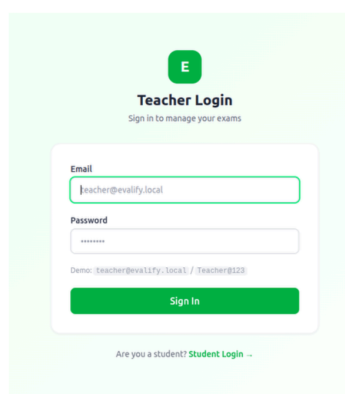
### Phase 0 — Project Setup:

### Phase 1 — Frontend Shell:

Our project has mainly two view: teacher and student view

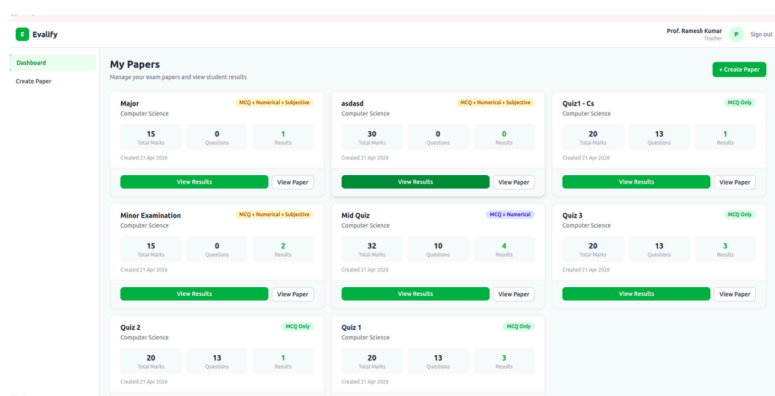
Login page

(credentials will be assigned by institute)



The login page features a green header with the Evalify logo. Below it, a 'Teacher Login' section prompts the user to sign in to manage exams. It includes input fields for 'Email' (pre-filled with 'teacher@evalify.local') and 'Password' (masked with dots). A demo credential is shown: 'Demo: teacher@evalify.local / Teacher@123'. A green 'Sign In' button is at the bottom. A link for 'Student Login' is provided for non-teacher users.

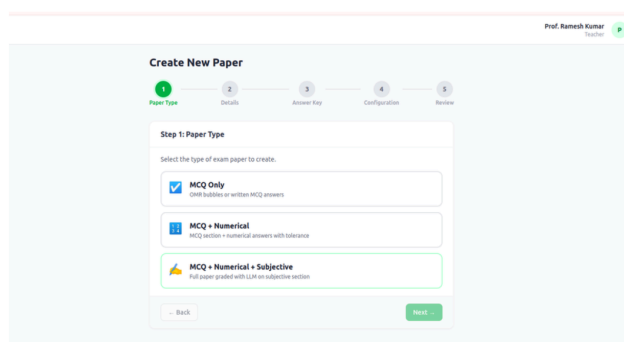
Teacher Dashboard



The dashboard shows a grid of 'My Papers' with columns for paper name, total marks, questions, and results. Each paper has a 'View Results' and 'View Paper' button.

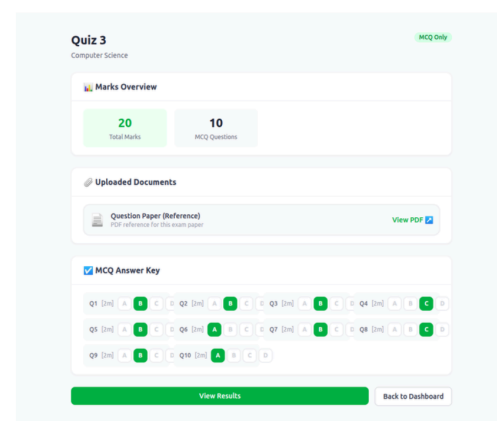
| Paper Name                         | Total Marks | Questions | Results |
|------------------------------------|-------------|-----------|---------|
| Major Computer Science             | 15          | 0         | 1       |
| Minor Examination Computer Science | 15          | 0         | 2       |
| Quiz 2 Computer Science            | 20          | 13        | 1       |
| asdasd Computer Science            | 30          | 0         | 0       |
| Mid Quiz Computer Science          | 32          | 10        | 4       |
| Quiz 1 Computer Science            | 20          | 13        | 3       |
| Quiz 1 - Cs Computer Science       | 20          | 13        | 1       |
| Quiz 3 Computer Science            | 20          | 13        | 3       |

Create a new paper (Three Types Allowed)



The 'Create New Paper' form has a progress bar with five steps: Paper Type, Details, Answer Key, Configuration, and Review. In the 'Paper Type' step, users select the exam type. Three options are available: 'MCQ Only' (radio button), 'MCQ + Numerical' (radio button), and 'MCQ + Numerical + Subjective' (radio button). Each option has a brief description of the paper type.

Individual Paper View



The 'Individual Paper View' for 'Quiz 3' shows a 'Marks Overview' section with 'Total Marks' (20) and 'MCQ Questions' (10). Below this is the 'Uploaded Documents' section, which includes a 'Question Paper (Reference)' and an 'MCQ Answer Key'. The answer key is a table with columns for question numbers and their corresponding answers.

| Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 | Q8 | Q9 | Q10 |
|----|----|----|----|----|----|----|----|----|-----|
| A  | B  | C  | D  | A  | B  | C  | D  | A  | B   |

Result view: shows the result of the Students: In future will be consist of several graphs: example normal distribution: which can be used to visualize and analyze performance of class

[-- Back to Dashboard](#)

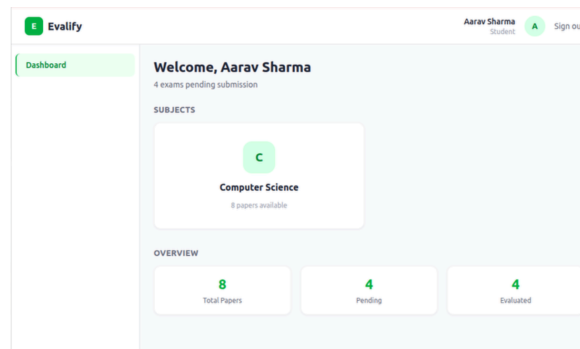
### Results — Mid Quiz

|                  |                       |                     |                    |
|------------------|-----------------------|---------------------|--------------------|
| 4<br>Submissions | 30.8<br>Average Score | 32<br>Highest Score | 27<br>Lowest Score |
|------------------|-----------------------|---------------------|--------------------|

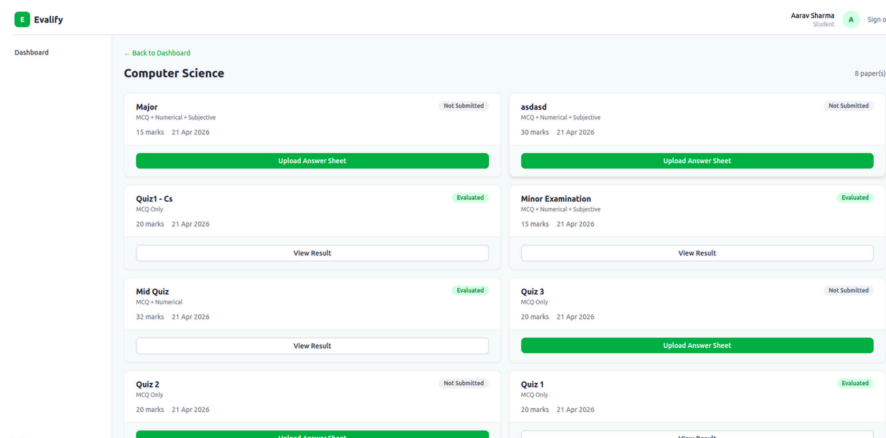
| STUDENT NAME | ROLL NO   | SCORE   | PERCENTAGE | GRADE | SUBMITTED             |
|--------------|-----------|---------|------------|-------|-----------------------|
| Priya Patel  | CS2025002 | 32 / 32 | 100.0%     | A+    | 21 Apr 2026, 05:03 am |
| Aarav Sharma | CS2025001 | 32 / 32 | 100.0%     | A+    | 21 Apr 2026, 05:16 am |
| Ananya Singh | CS2025004 | 32 / 32 | 100.0%     | A+    | 21 Apr 2026, 05:23 am |
| Rohit Verma  | CS2025003 | 27 / 32 | 84.4%      | A     | 21 Apr 2026, 05:37 am |

Student View:

### Student Dashboard



### Subject test

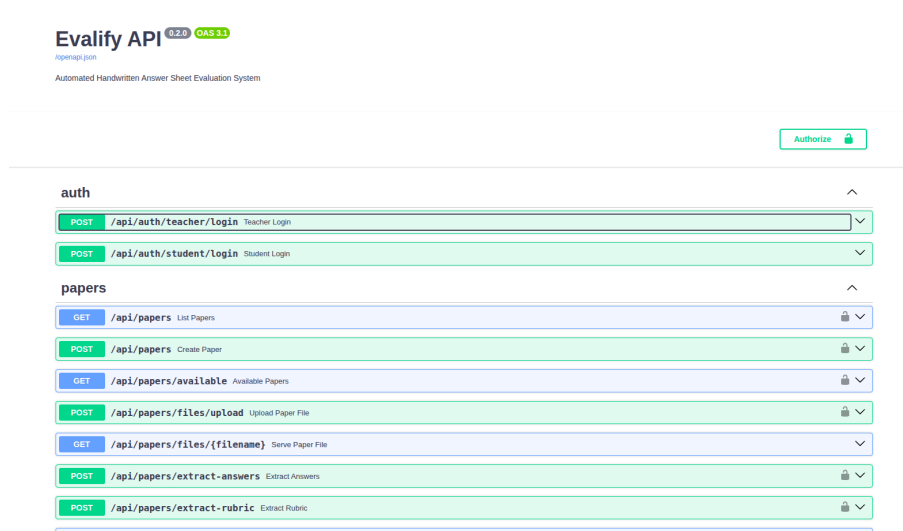


## Phase 2 — Backend Foundation

MongoDB: container is used , for authentication, storing the data of the students

Initially we have created 6 dummy students their details are stored in MongoDB database, it is also used for Authentication

SwaggerDocs: is used to Visualize and for detail documentation of all the exposed APIs



### Phase 3 – Type 1: Question Paper implemented (MCQs)

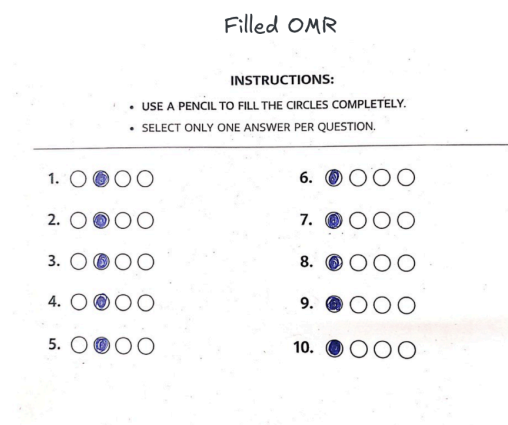
MCQs solution can be given in form of OMR or normal hand written answer sheet

Hence two approach is used to handle both cases

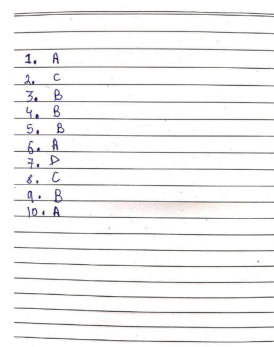
- To read the Students OMR we use Open CV OMR engine is to handle the simple cases, they give very fast response
- For the handwritten answersheet we have use local hosted Ollama 11b model, earlier we decided to use TrOcr model, but it resulted in hallucination of words

Sample MCQs Question Paper and Answer Key

| MCQ Question Paper (10 Questions)                                                                                                        | Answer Key |
|------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1. What is the time complexity of binary search?<br>A) $O(n)$ B) $O(\log n)$ C) $O(n \log n)$ D) $O(1)$                                  | 1. B       |
| 2. Which data structure uses FIFO principle?<br>A) Stack B) Queue C) Tree D) Graph                                                       | 2. B       |
| 3. What does CPU stand for?<br>A) Central Process Unit B) Central Processing Unit C) Computer Processing Unit D) Control Processing Unit | 3. B       |
| 4. Which language is primarily used for web development?<br>A) Python B) C++ C) JavaScript D) Java                                       | 4. C       |
| 5. What is the value of $2^{13}$ ?<br>A) 4 B) 8 C) 9 D) 4                                                                                | 5. B       |
| 6. Which of the following is not a programming language?<br>A) HTML B) Python C) Java D) C++                                             | 6. A       |
| 7. What is the full form of RAM?<br>A) Read Access Memory B) Random Access Memory C) Run Access Memory D) Real Access Memory             | 7. B       |
| 8. Which sorting algorithm is fastest on average?<br>A) Bubble Sort B) Selection Sort C) Quick Sort D) Insertion Sort                    | 8. C       |
| 9. What is the output of $5 \% 2$ ?<br>A) 2 B) 1 C) 0 D) 5                                                                               | 9. B       |
| 10. Which of the following is an operating system?<br>A) Windows B) Python C) HTML D) SQL                                                | 10. A      |



Handwritten Answersheet



### Phase 4 - Type 2 MCQ + Numerical Question Paper

- Main crux of this type of question paper was that the Numerical Answers can have more than one solution
- Lets see a template prompt send to the ollama model to understand the Student Answer sheet

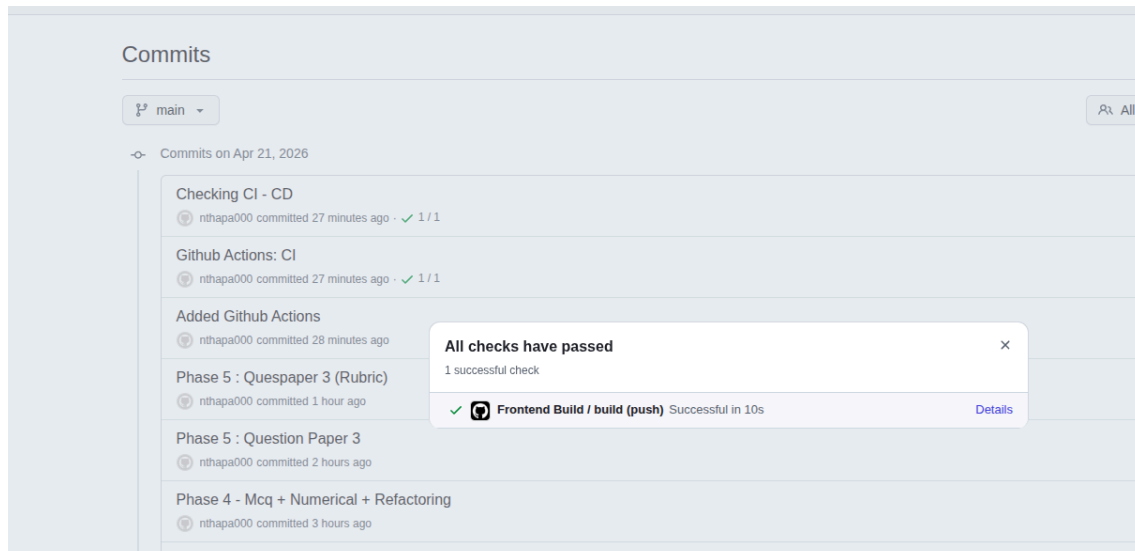
```
5 ) -> str:
6     """
7     Build the combined extraction prompt for a Type 2 answer sheet.
8
9     Students write a single answer sheet with all questions numbered Q1..Q_total.
10    - Q1 ... Q{mcq_count} → MCQ section (letter answers)
11    - Q{mcq_count+1} ... Q{mcq_count+numerical_count} → Numerical section (number/expression answers)
12
13    The prompt uses this natural Q-numbering so Ollama matches what it sees on the sheet.
14    """
15    total = mcq_count + numerical_count
16    num_start = mcq_count + 1
17
18    return f"""You are reading a student's answer sheet with {total} questions total.
19
20    Questions Q1 to Q{mcq_count} are MCQ questions – answers are letters ({options}).
21    Questions Q{num_start} to Q{total} are Numerical questions – answers are numbers or expressions.
22
23    Rules for MCQ (Q1-Q{mcq_count}):
24    - Single option written (e.g. "B") → value is "B"
25    - Multiple options written (e.g. "A" and "C") → join sorted with comma: "A,C"
26    - Blank or unreadable → omit that key
27
28    Rules for Numerical (Q{num_start}-Q{total}):
29    - Copy the number or expression exactly as written (e.g. "3", "3.0", "1/2", "0.5")
30    - If two values are written separated by comma (e.g. "5/8, 0.625") take the FIRST one
31    - Blank or unreadable → omit that key
32
33    Return ONLY a JSON object – no explanation, no preamble.
34
35    {{
36    "mcq": {{ "Q1": "C", "Q2": "B", ..., "Q{mcq_count}": "B" }},
37    "numerical": {{ "Q{num_start}": "3.5", "Q{num_start+1}": "1.5", ..., "Q{total}": "3" }}
38    }}
39
40    Now extract all answers from the image."""
```

### Phase 5 - Type 3 MCQ, Numerical and Subjective Question Paper:

- Here we implemented a LLM subjective evaluation by doing Prompt engineering and sending high quality prompts as request to locally hosted LLM
- We mainly had two main part of the prompt send for evaluation which was
- Grade Rubric and Reference Answer (Answer referred by LLM for Evaluation)

### Phase 6 - Github Actions and Docker containerization

- Implemented simple but very essential checks like npm run build and boosted continuous integration of the code (CI-CD)



- Containerized every component

## 4. Limitation

---

- Locally runned LLMs like ollama:11b can only ensure the accuracy upto a certain extent, larger the model used better the accuracy
- Worse the handwriting of the student harder it becomes for LLM to understand it, and accuracy decreases by a lot

## 5. Github and Demo Link

---

**Github:** <https://github.com/nthapa000/Evalify>

**Drive Link:** [Evalify.mp4](#)